Data warehousing

Data models in database systems

Lecture 5

Mara Romanovska
Mara.Romanovska@rtu.lv

23.10.2025.

RTU
DATORZINĀTNES UN
INFORMĀCIJAS
TEHNOLOĢIJAS FAKULTĀTE

Agenda

- About PW1
- Data analysis as a task
- Multidimensional data model
- Designing multidimensional data model as a relational schema

Two important comments regarding PW1

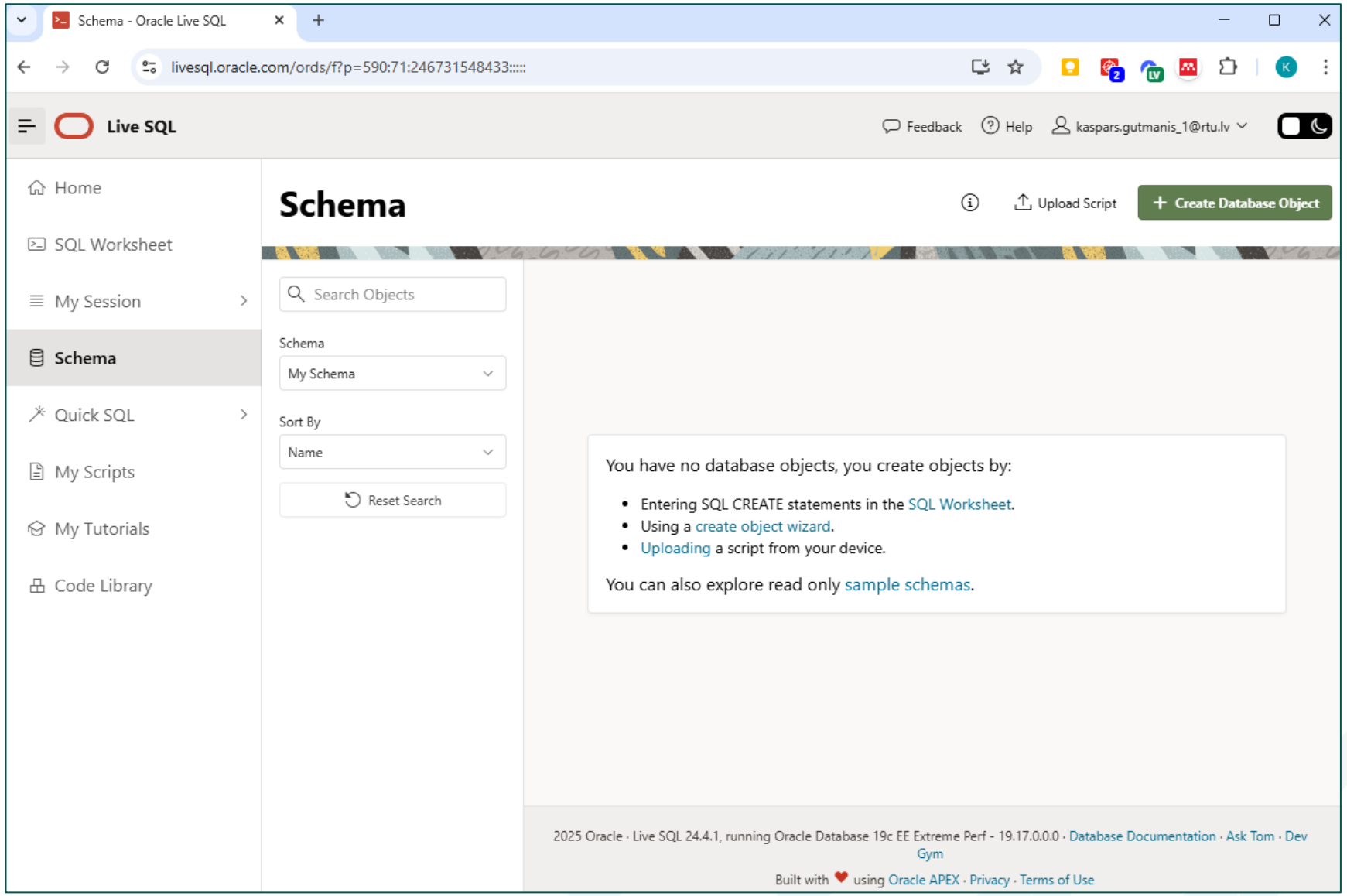
1. Please note the diagrams look and are formatted accordingly
   - Font is the size 10-12 and diagram fits in one page
   - No dark background
   - ORTUS distorts PDFs and they become unreadable; it also shows the amount of work to do
   - **THEY WILL NOT BE GRADED OTHERWISE without the option to resubmit**
2. Make sure your script is formatted properly
   - Script must be formatted as shown is the links; we have added a tutorial on how to do this
   - The works that are **uncheckable** (does not contain project passport AND clearly indicated **statement** numbers on the **correctly formatted executed script**) **WILL NOT BE GRADED without the option to resubmit**

How to upload and share your script using LiveSQL Classic?

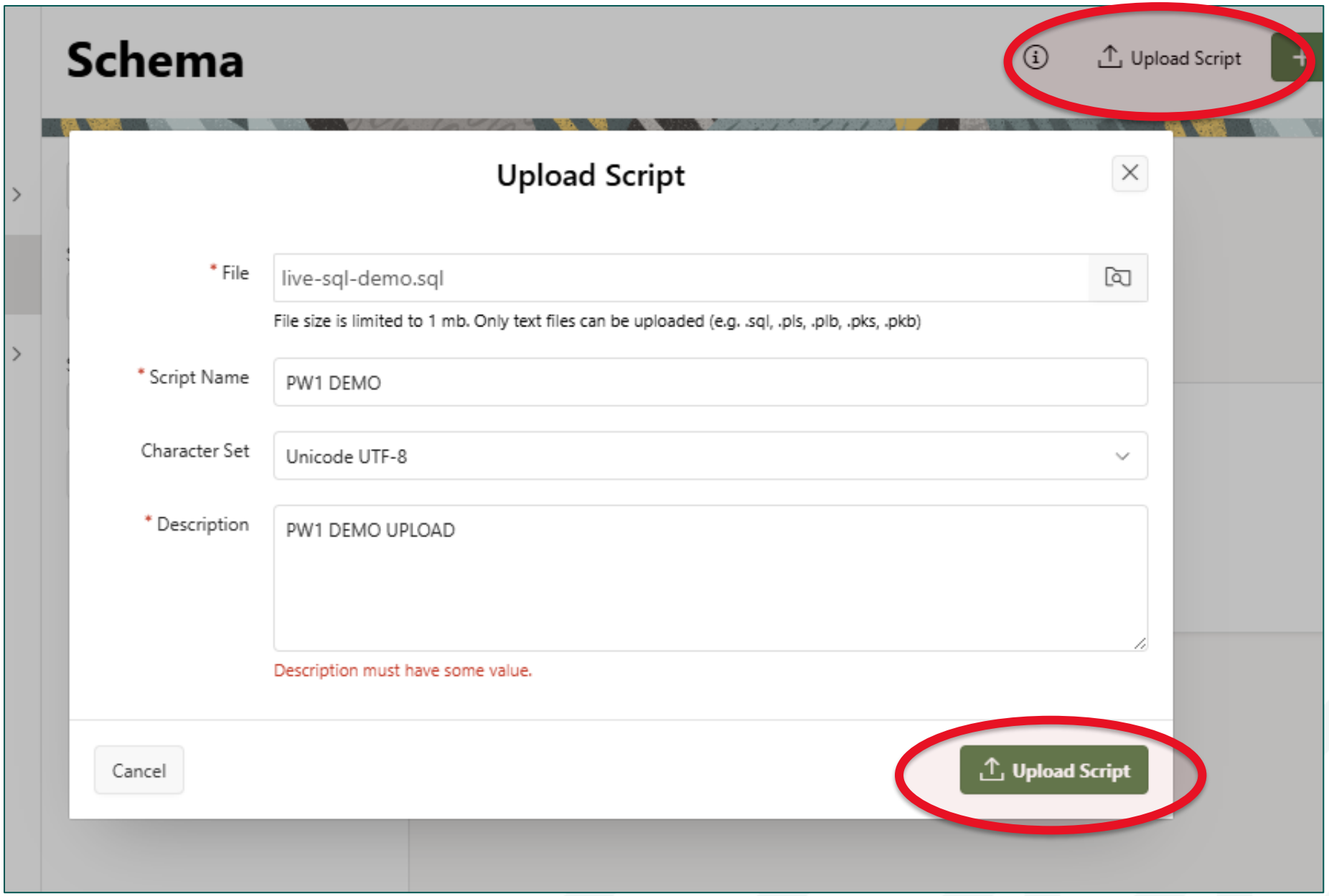
1862
RTU
DATORZINĀTNES UN
INFORMĀCIJAS
TEHNOLOĢIJAS FAKULTĀTE

Ensure that your database is empty

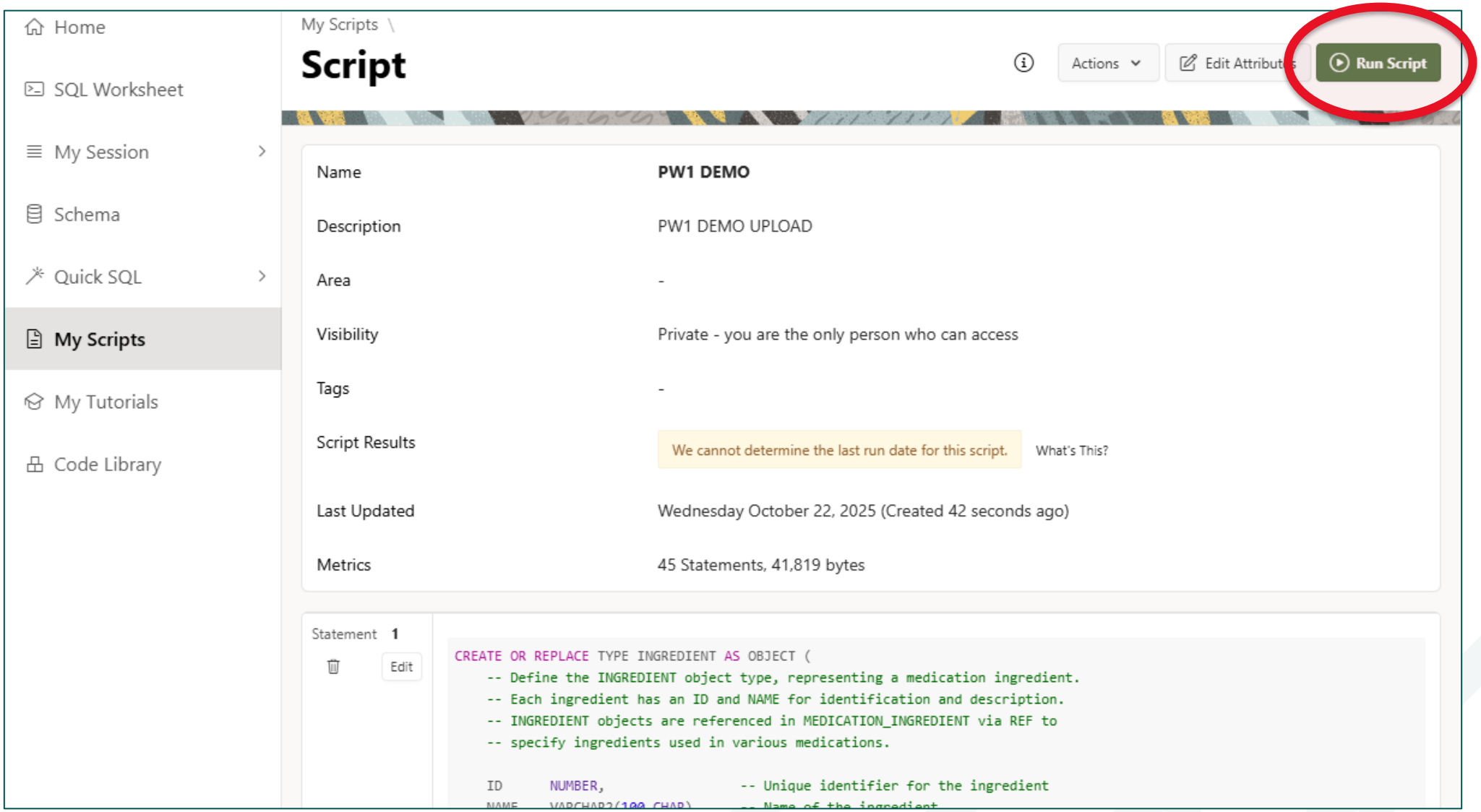
(My Schema)



Upload your script

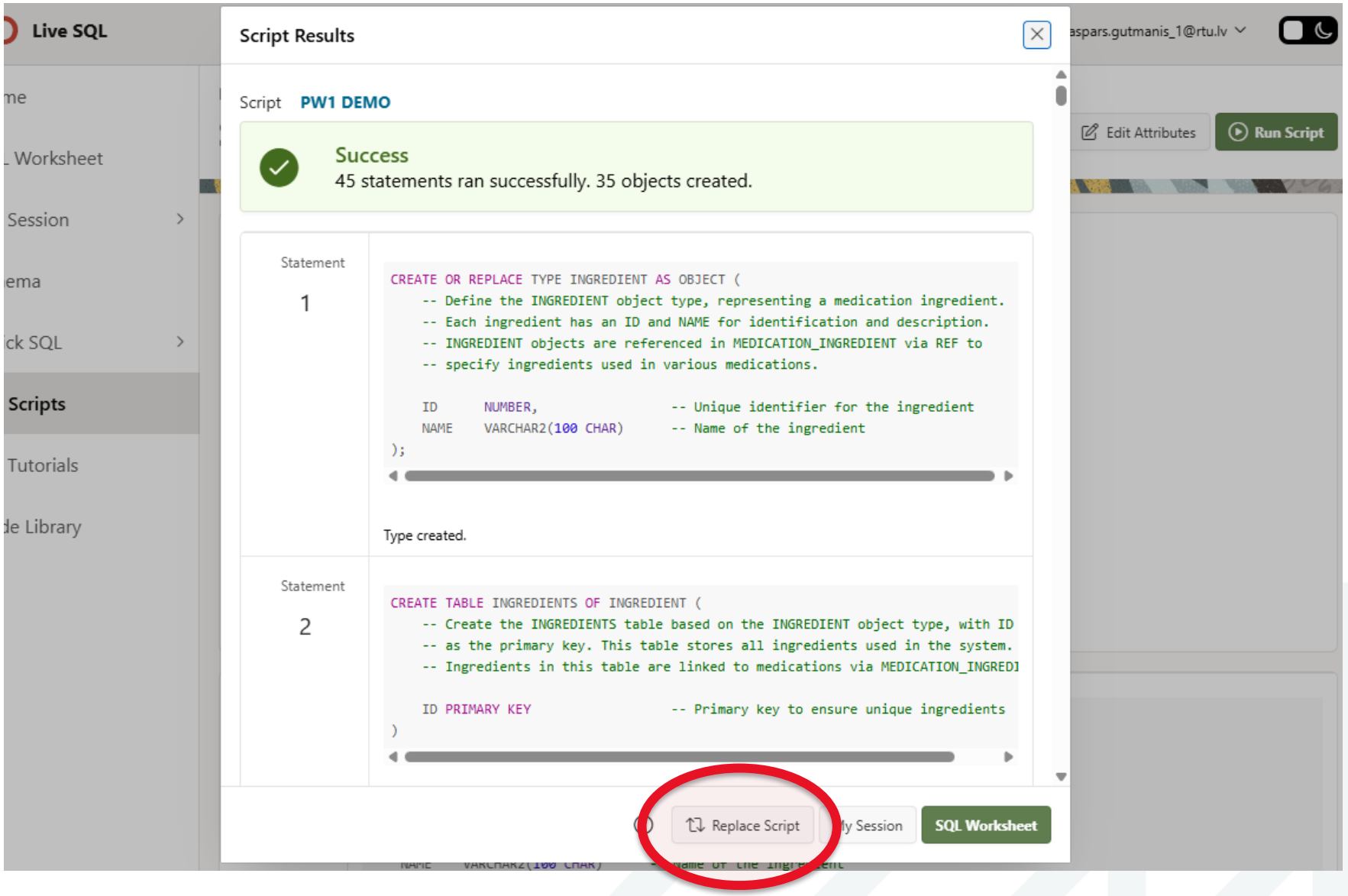


Run your script



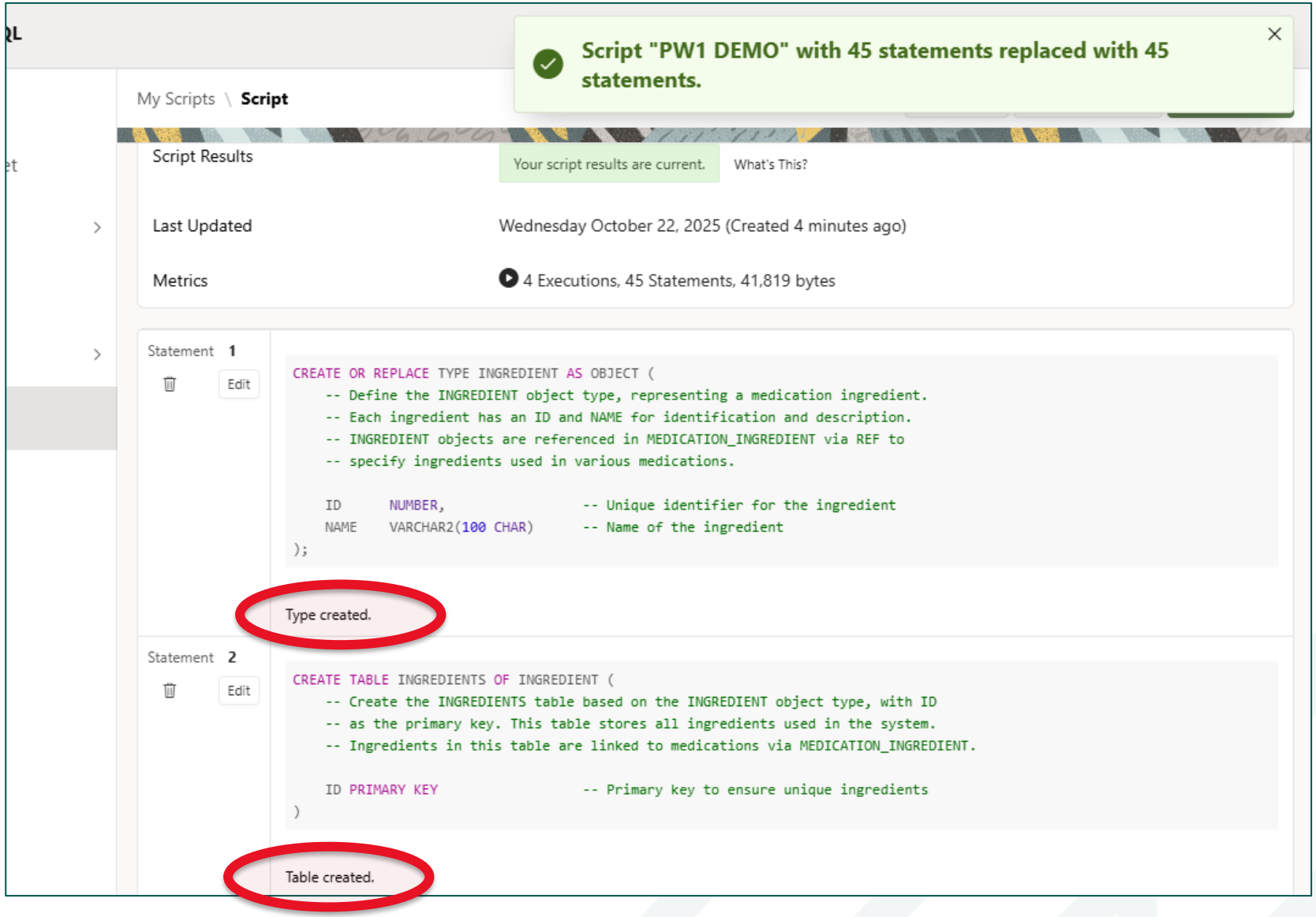
Replace your script after running it!

That's how you ensure that all the script output is saved!

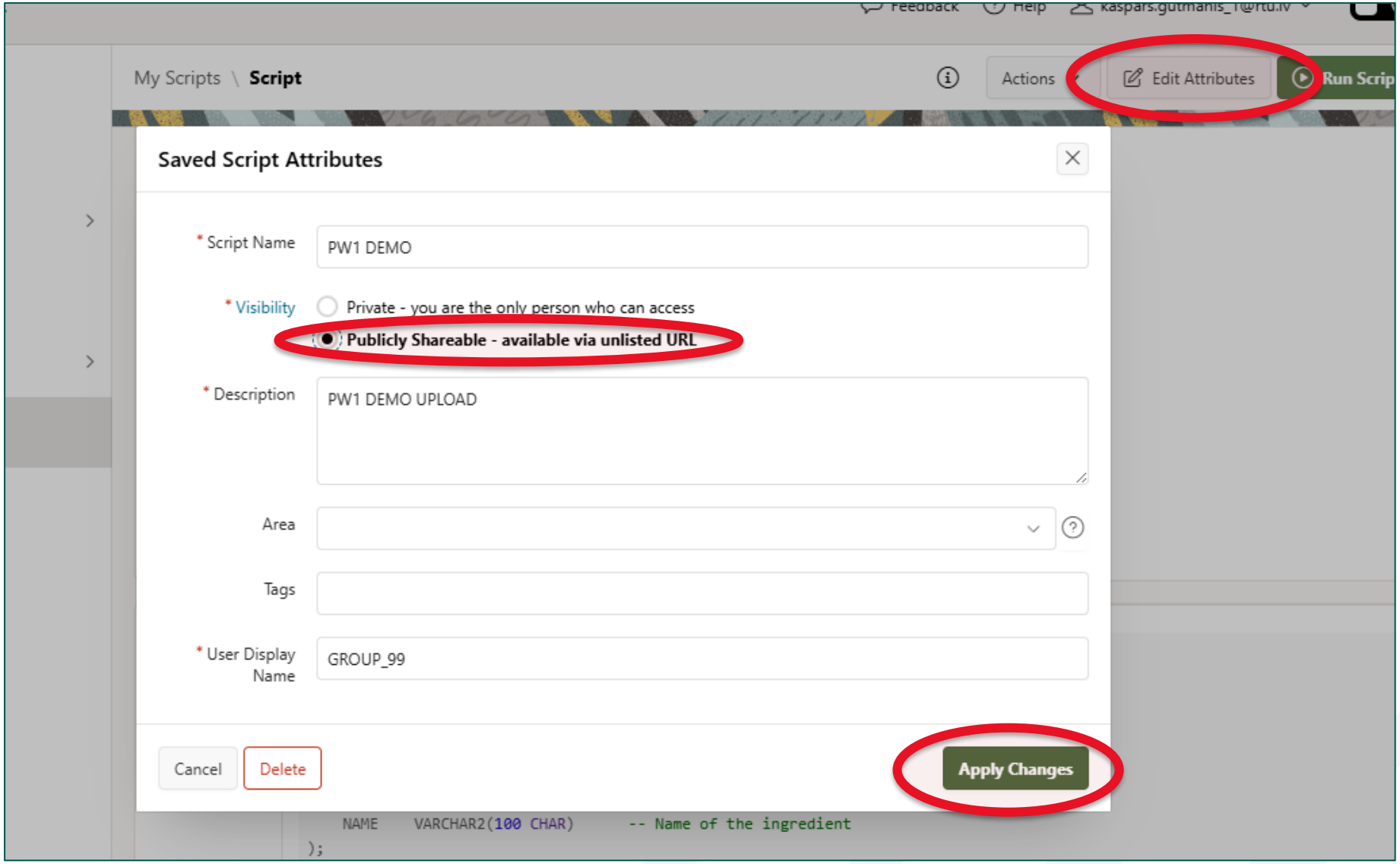


That’s what you should see after previous step

Output or query result must be visible after each statement!



Create shareable link by editing script attributes!

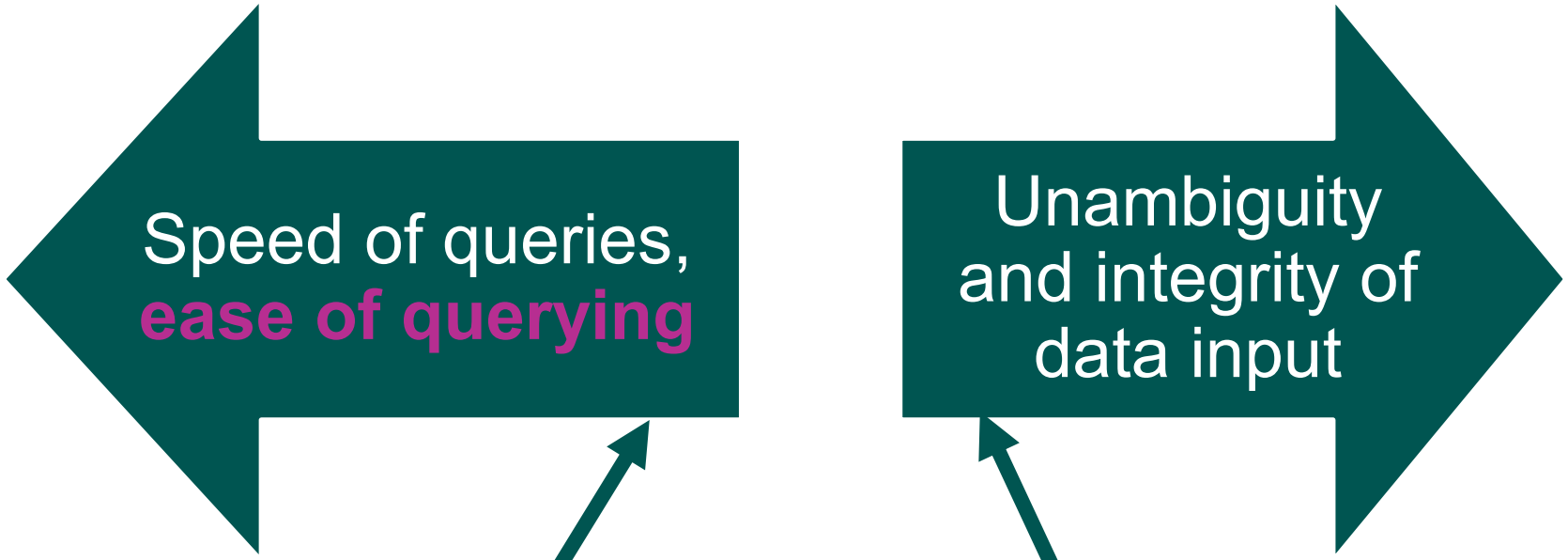


Analytics «mode» of databases

How to use the data?

- Queries and reports (using the database for general business support) – **OLTP**
  - Processing **transactions**
- Get either more detailed information, or more general information – data analytics, **OLAP**
  - **Data warehousing** technologies
- Extracting patterns and generalizations – **data science**
  - Machine learning techniques

Database dilemmas



- **Data warehouses** are database systems used for data analysis
- **Solutions** to transform operational data into a format suitable for analysis

- **Operational databases** - basically the database systems we've talked about so far

Data warehouse

- A **data warehouse** (DW) is a type of the database used **for reporting**
- A **data warehouse** is a database specifically structured for **query and analysis**
- A data warehouse typically contains data representing the business history of an organization
- The concept of data warehousing has evolved out of the need for easy access to a structured store of quality **data that can be used for decision making**

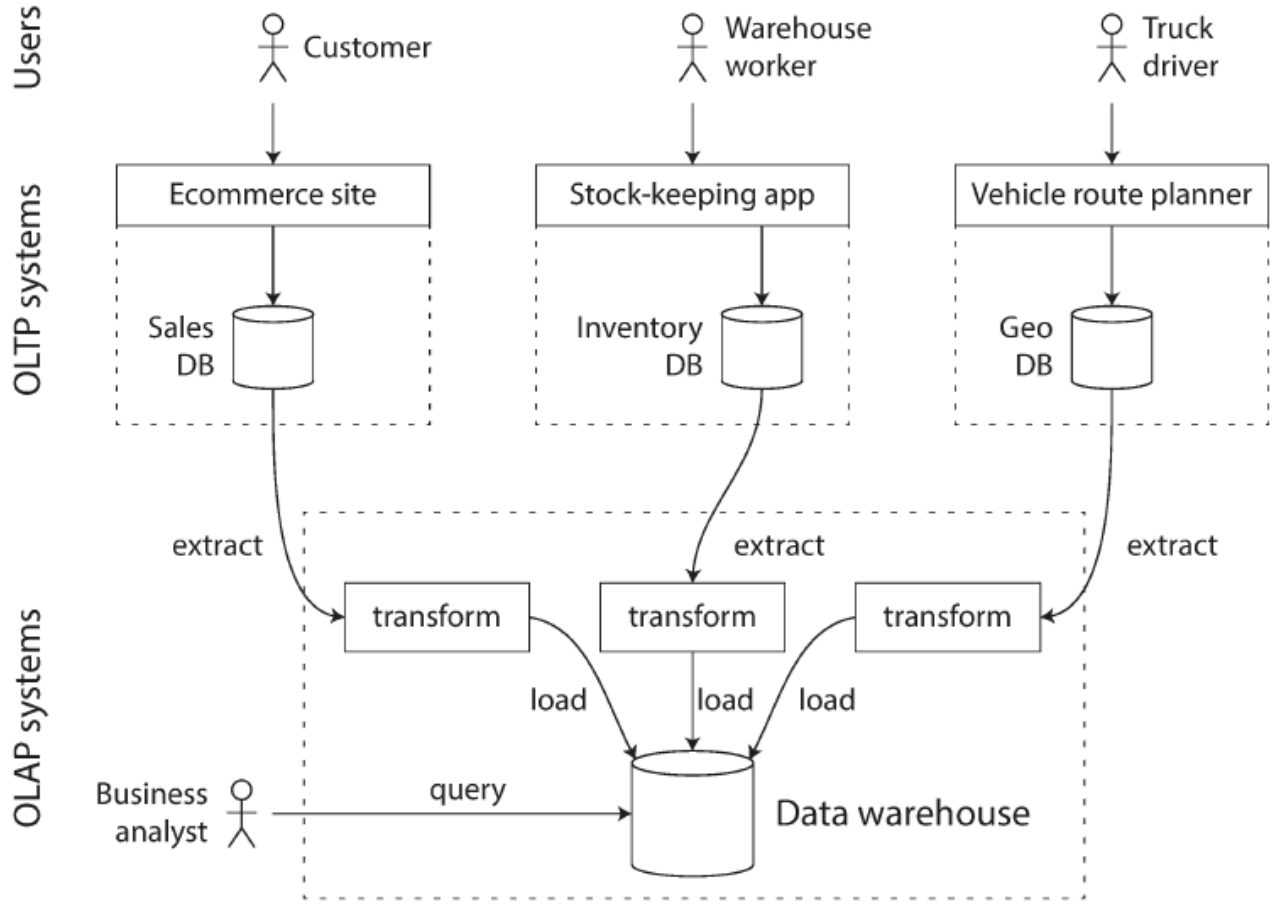
Online analytical processing (OLAP)

- **OnLine Analytical Processing (OLAP)** is an approach to swiftly answer **multi-dimensional** analytical queries.
- Databases configured for OLAP use a **multidimensional data model**, allowing for complex analytical and ad-hoc queries with a rapid execution time.
- The output of an OLAP query is typically displayed in a matrix (or pivot) format. The dimensions form the rows and columns of the matrix; the measures form the values.

OLTP vs. OLAP

OLAP	OLTP
Subject oriented	Transaction oriented
Large (hundreds of GB up to several TB)	Small (MB up to several GB)
Historic data	Current data
De-normalized table structure (few tables, many columns per table)	Normalized table structure (many tables, few columns per table)
Batch updates	Continuous updates
Usually very complex queries	Simple to complex queries

Warehouse creating process

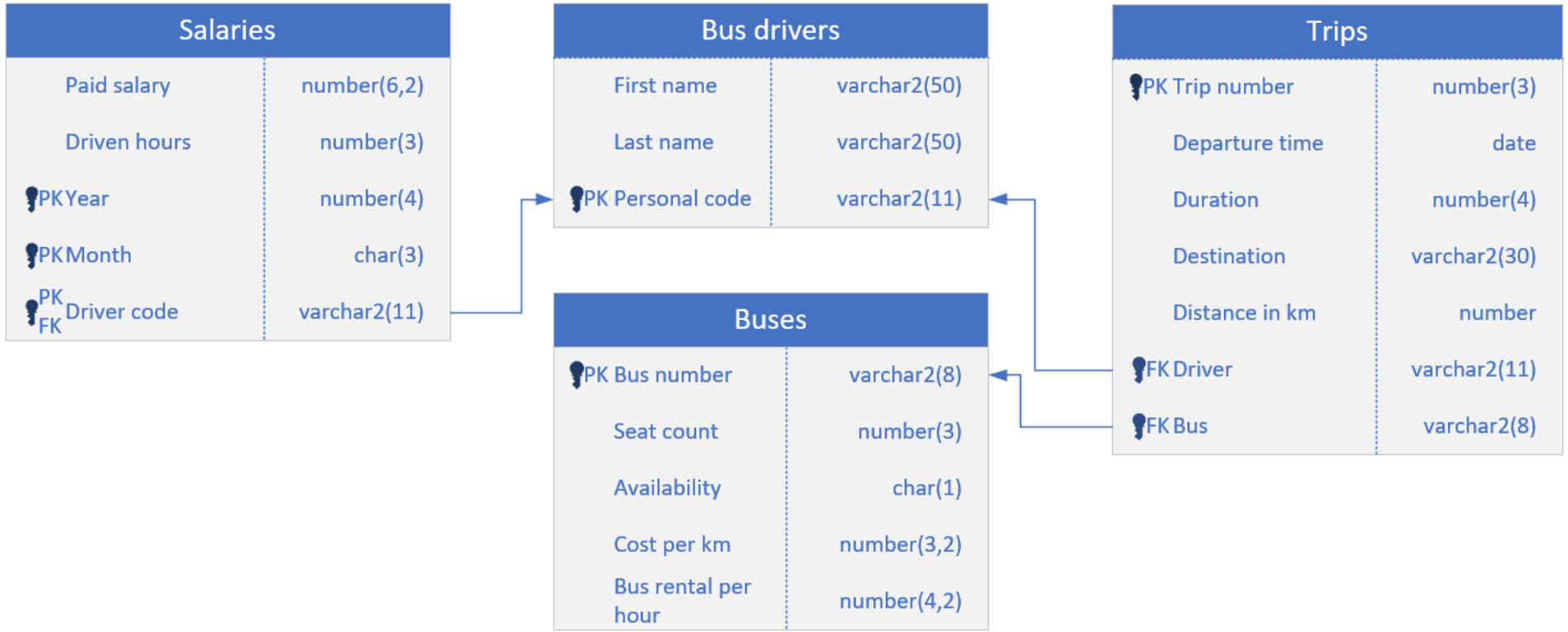


ETL process

- **ETL (Extract, Transform and Load)** is a process in data warehousing responsible for pulling data out of the source systems and placing it into a data warehouse

- ETL involves the following tasks:
  - extracting the data from source systems
  - transforming the data may involve the following tasks:
    - applying business rules
    - cleaning
    - filtering
    - splitting a column into multiple columns and vice versa,
    - joining together data from multiple sources
    - transposing rows and columns,
    - applying any kind of simple or complex data validation
  - loading the data into a data warehouse or data repository other reporting applications.

- Suppose we need to frequently retrieve **aggregated values** from a database and perform calculations with these values
- Option (a):
  - Multiple SELECT queries, performs calculations on the application side
- Option (b)
  - **A dedicated solution that retrieves or stores these aggregated values on the database side**



Different kinds of OLAP systems

- **Multidimensional OLAP (MOLAP)**
  - the 'classic' form of OLAP
  - MOLAP stores this data in **an optimized multi-dimensional array storage, rather than in a relational database**
  - requires the pre-computation and storage of information in the cube - the operation known as processing

- **Relational OLAP (ROLAP)**
  - ROLAP works directly with relational databases.
  - The base **data and the dimension tables are stored as relational tables and new tables are created to hold the aggregated information**

- **Hybrid OLAP (HOLAP)**
  - Database will divide data between relational and specialized storage

Multi-dimensional data model

The challenge we are trying to tackle

- Let's imagine that we often need to: calculate the **number of** different **trips** based on **bus drivers, buses and destinations**

BUS_NUMBER	'Berlin'	'Madrid'	'Poznan'	'Rome'	'Prague'
1L1K3	0	2	0	1	0
5Y573M5	2	0	3	0	3
D474B453	3	0	1	1	2

Pivot tables

- Tables of this type, where the values of the fields are taken and aggregated into the cells of the table, are called **pivot tables**
- **Rows and columns contain field values**

TRIP_NUMBER	DEPARTURE_TIME	DURATION	DESTINATION	KILOMETERS	DRIVER	BUS_NUMBER
111	02-JAN-23	74	Madrid	7176	67390282019	1L1K3
112	16-JAN-23	54	Rome	4858	67390282019	D474B453
113	05-DEC-22	30	Berlin	2448	67390282019	5Y573M5
114	05-DEC-22	30	Prague	2606	12436190190	D474B453
115	12-DEC-22	28	Berlin	2478	12436190190	D474B453
116	09-JAN-23	28	Berlin	2478	12436190190	D474B453
117	12-DEC-22	22	Poznan	1965	56129018212	5Y573M5
118	19-DEC-22	30	Prague	2606	56129018212	5Y573M5

BUS_NUMBER	'Berlin'	'Madrid'	'Poznan'	'Rome'	'Prague'
1L1K3	0	2	0	1	0
5Y573M5	2	0	3	0	3
D474B453	3	0	1	1	2

Oracle DBMS pivot tables with PIVOT clause

- Use the PIVOT clause to specify
  - Which fields will make up the pivot table
  - What will be the aggregation function

```
SELECT * FROM trip_statistics
PIVOT(
    COUNT(trip_number)
    FOR destination
    IN ('Berlin', 'Madrid', 'Poznan', 'Rome', 'Prague')
)
ORDER BY bus_number;
```

- Note the requirements for the table "trip_statistics" - it **consists of the row/column fields in the cross table and the aggregate field**

The challenge we are trying to solve - we need a 3rd dimension - or we need a data cube

- Let's imagine that we often need to: calculate the **number of** different **trips** based on **bus drivers, buses and destinations**

① ② ③

BUS_NUMBER	'Berlin'	'Madrid'	'Poznan'	'Rome'	'Prague'
1L1K3	0	2	0	1	0
5Y573M5	2	0	3	0	3
D474B453	3	0	1	1	2

Aggregated values by driver, bus number, destination

Driver 3

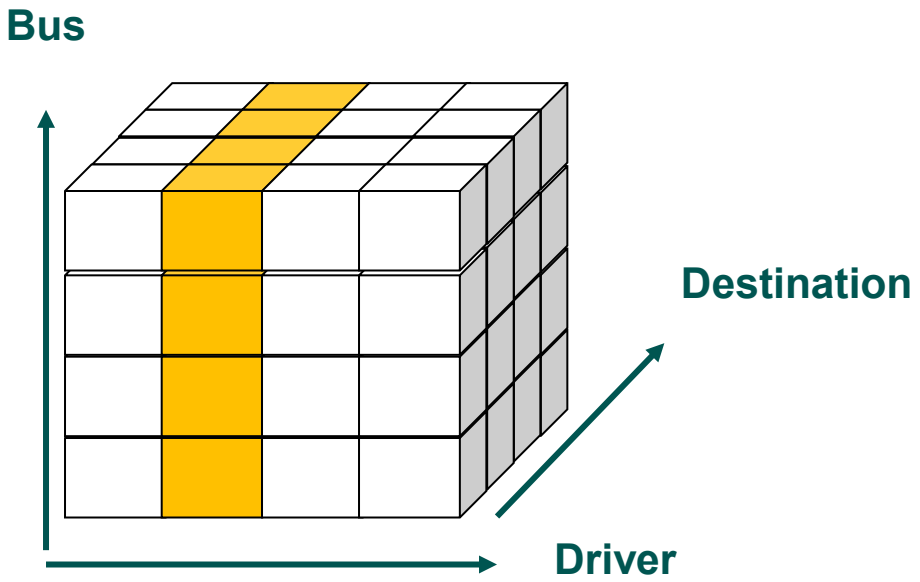
Driver 2

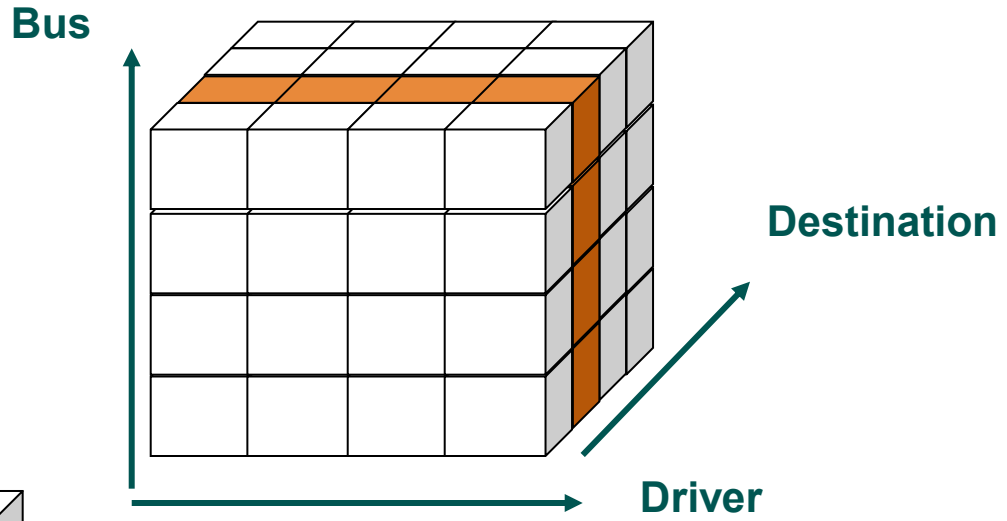
Driver 1

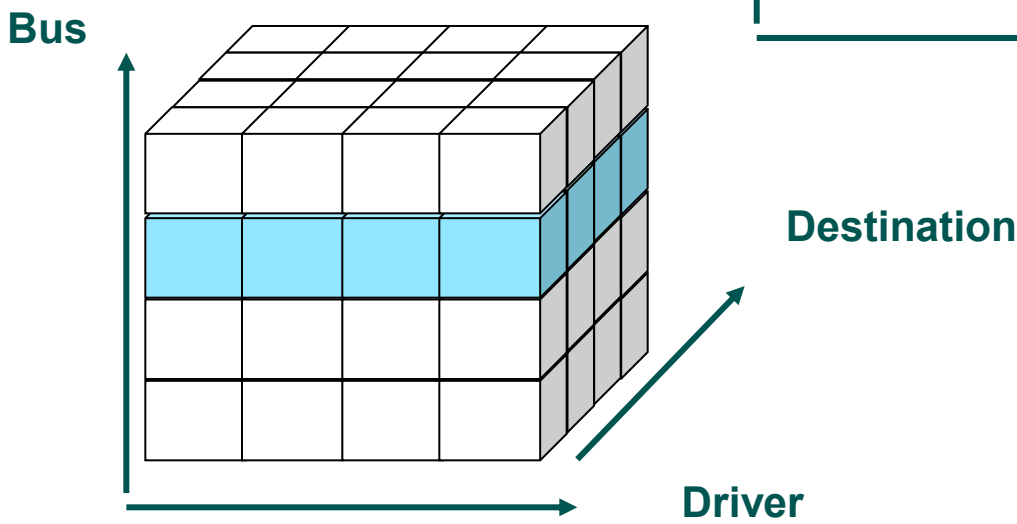
Data cube components

- Destinations, bus drivers and buses are the **dimensions** or aspects to be analysed
- **Hierarchies** can form within dimensions
  - Time dimension: months and years
- The result or aggregation function is a **measure**
- Dimension values + measure = **facts**

Slices of a data cube







Reference to cells

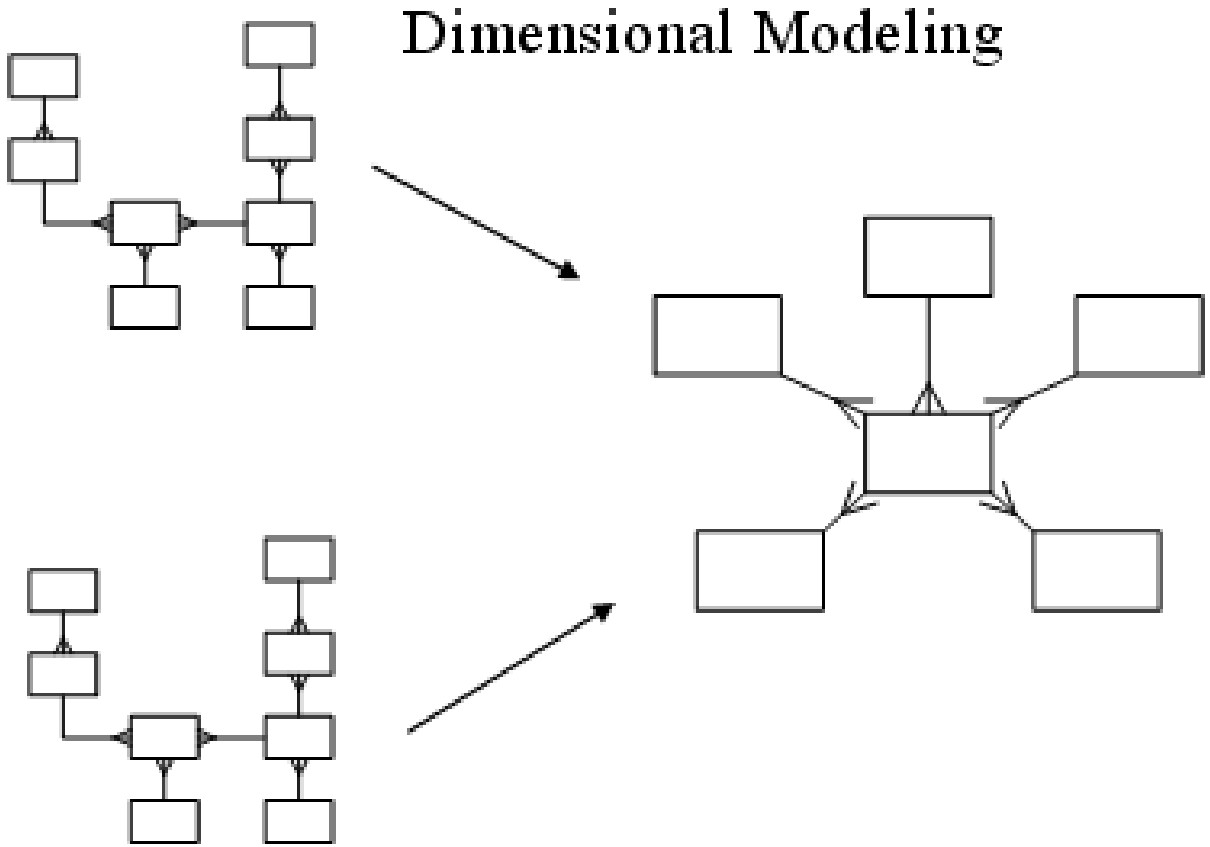
- There are two ways to refer: symbolically or positionally
- You can also "build formulas" with the help of invocations and various predicates, i.e. get slices

Trip_number [bus_number = ‘1L1K3', destination = 'Poznan’, bus_driver = '67390282019']

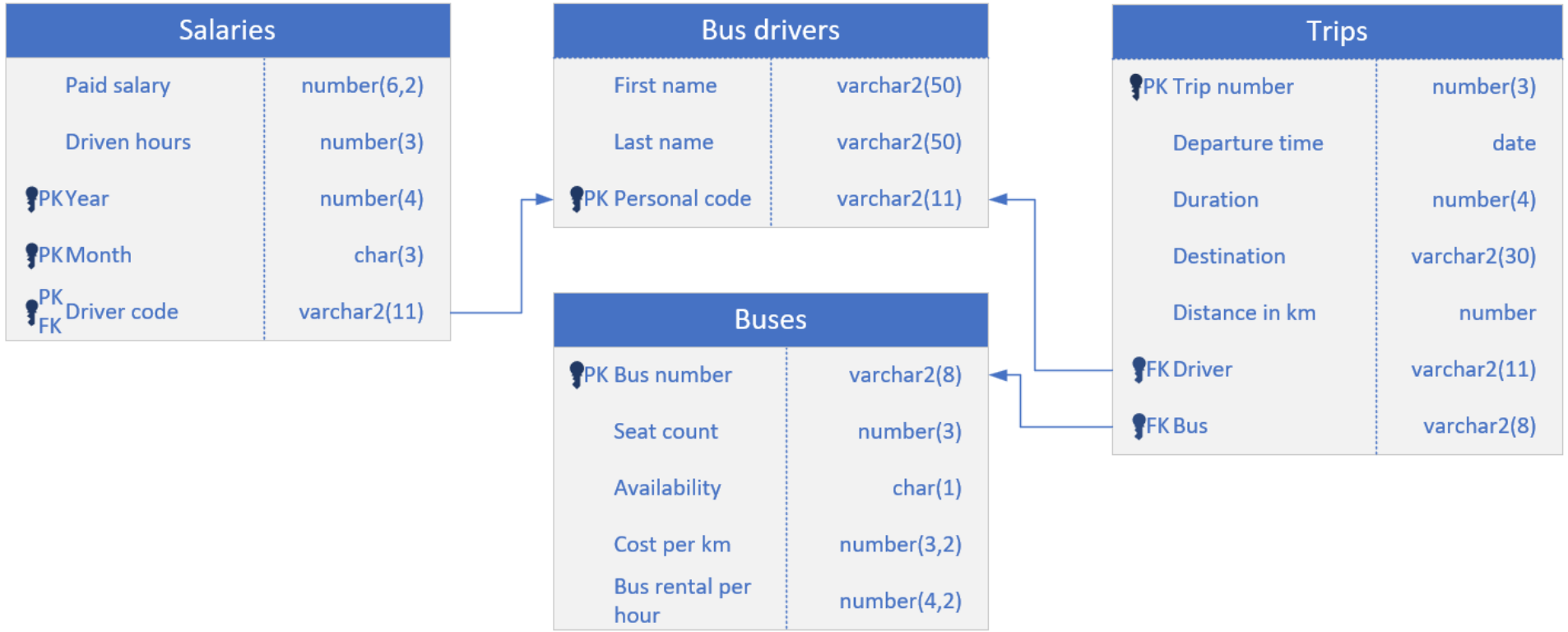
Trip_number[‘1L1K3 ', 'Poznan', '67390282019']

The design of multi-dimensional database

Dimensional modelling



- Suppose we need to frequently retrieve **aggregated values** from a database and perform calculations with these values
- Option (a):
  - Multiple SELECT queries, performs calculations on the application side
- Option (b)
  - **A dedicated solution that retrieves or stores these aggregated values on the database side**



Storing a multidimensional model in a relational DB

- Relations are stored in a fact table - the structure of the fact table is similar to the structure required for a PIVOT clause

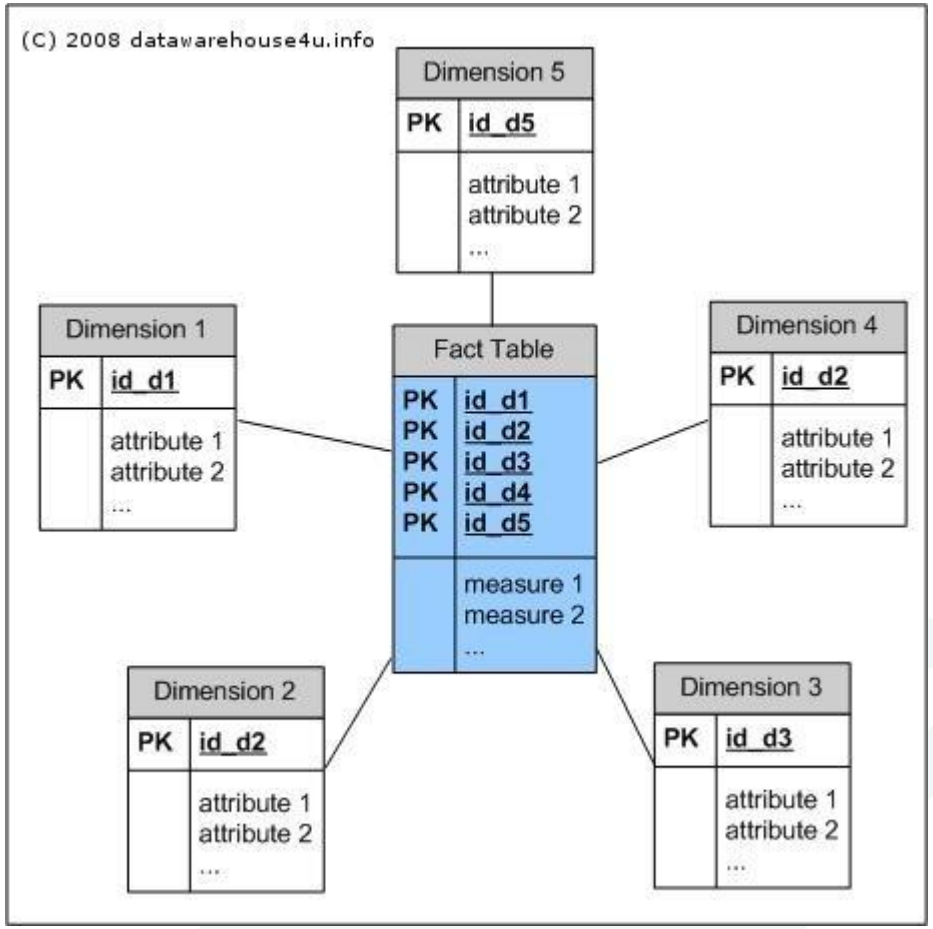
The combination of dimensions uniquely identifies the cell

DESTINATION	BUS_NUMBER	DRIVER	COUNT
Prague	D474B453	12436190190	1
Berlin	D474B453	64792181931	1
Madrid	1L1K3	67390282019	1
Madrid	1L1K3	98614209810	1
Berlin	5Y573M5	67390282019	1
Prague	5Y573M5	56129018212	2

Values in cells, or measures

Star schema

- The simplest style of data warehouse schema.
- The star schema consists of **one or more fact tables** referencing any number of **dimension tables**



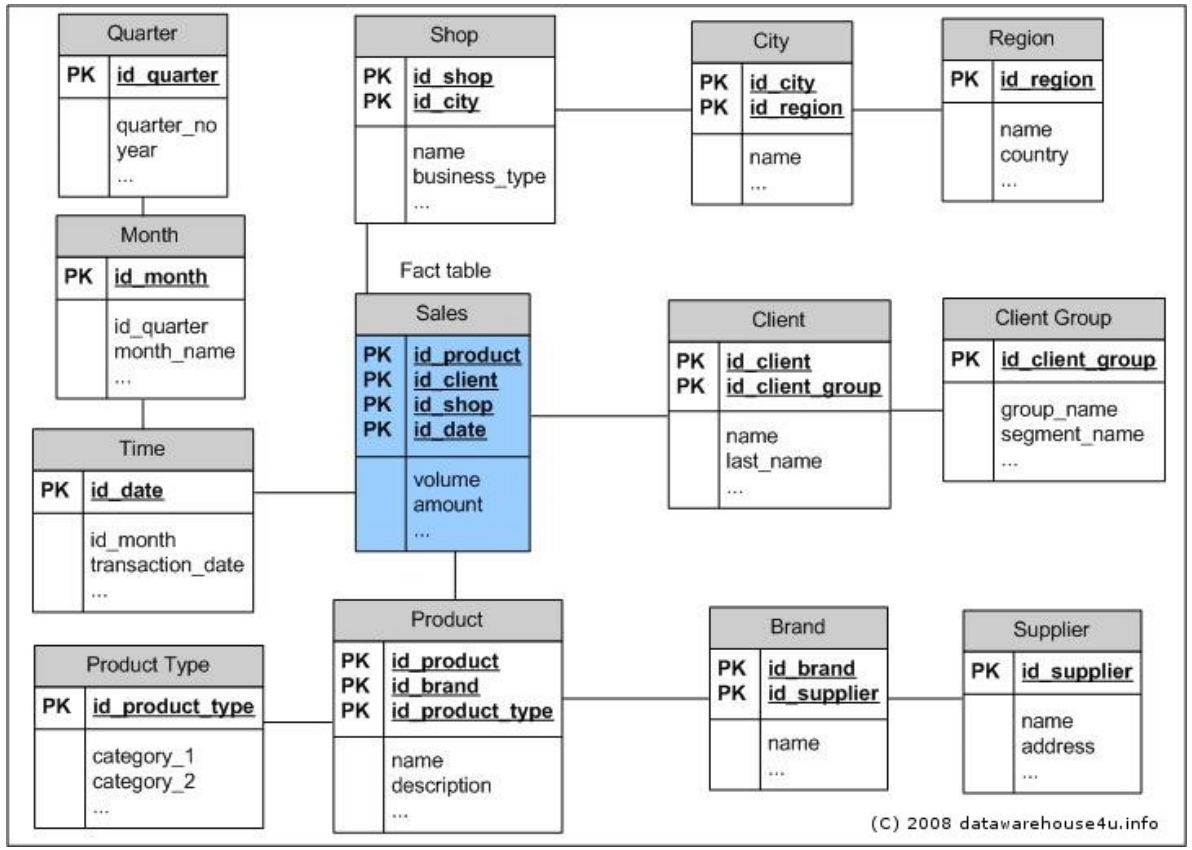
Dimension tables

- Dimension tables describe the **business entities of an enterprise**, represented as hierarchical, categorical information such as time, departments, locations, and products.
- Dimension tables are sometimes **called lookup or reference tables**.
- Dimension tables usually change slowly over time and are not modified on a periodic schedule.
- They are used in long-running decision support queries to aggregate the data returned from the query into appropriate levels of the dimension hierarchy.

Fact table

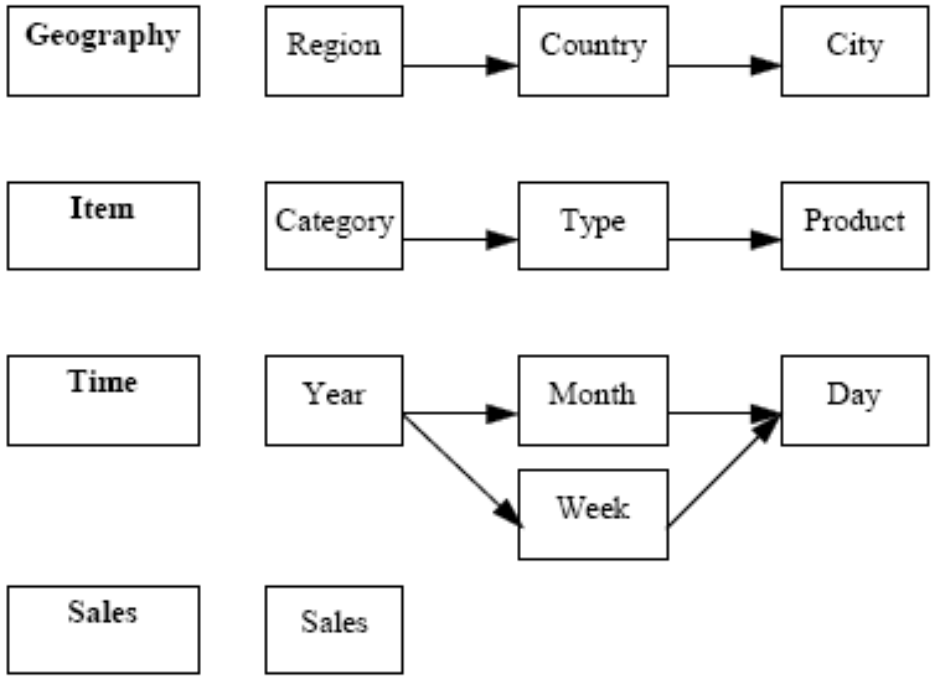
- Fact tables describe **the business transactions** of an enterprise.
- The vast majority of data in a data warehouse is stored in a few very large fact tables that are updated periodically with data from one or more operational OLTP databases.
- Fact tables include facts (also called **measures**) such as sales, units, and inventory.
  - A simple measure is a numeric or character column of one table
  - A computed measure is an expression involving measures of one table
  - A multitable measure is a computed measure defined on multiple tables
- Fact tables also contain one or more foreign keys that organize the business transactions by the relevant business entities such as time, product, and market.

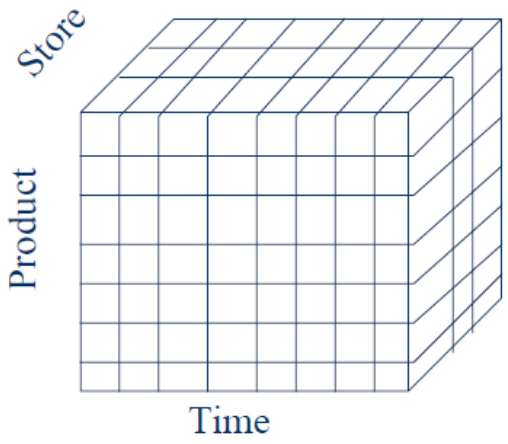
Snowflake


Quarter
PK id_quarter
quarter_no
year
...
Month
PK id_month
id_quarter
month_name
...
Time
PK id_date
id_month
transaction_date
...
Product Type
PK id_product_type
category_1
category_2
...
Shop
PK id_shop
PK id_city
name
business_type
...
Fact table
Sales
PK id_product
PK id_client
PK id_shop
PK id_date
volume
amount
...
Product
PK id_product
PK id_brand
PK id_product_type
name
description
...
City
PK id_city
PK id_region
name
...
Region
PK id_region
name
country
...
Client
PK id_client
PK id_client_group
name
last_name
...
Client Group
PK id_client_group
group_name
segment_name
...
Brand
PK id_brand
PK id_supplier
name
...
Supplier
PK id_supplier
name
address
...
(C) 2008 datawarehouse4u.info

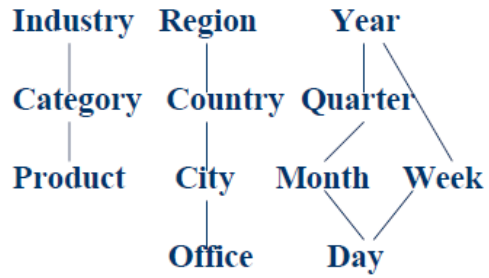
Dimension hierarchies and dimension levels

- Hierarchies describe the business relationships and common access patterns in the database.

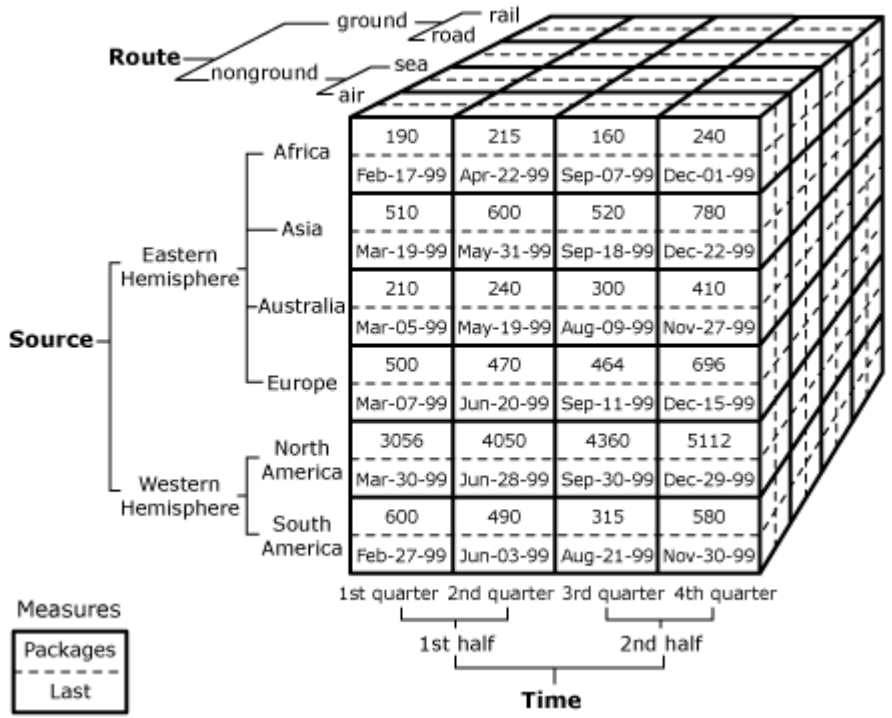


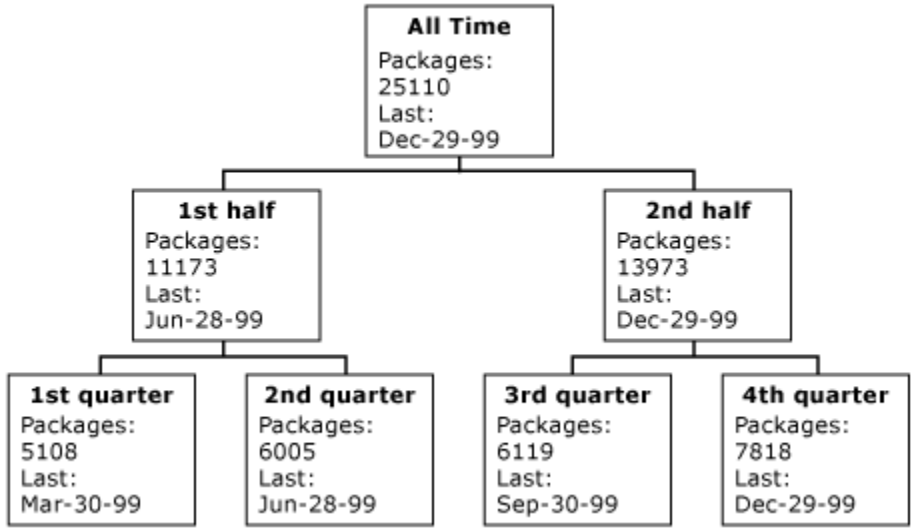


Dimensions: Product, Location, Time
Hierarchical summarization paths

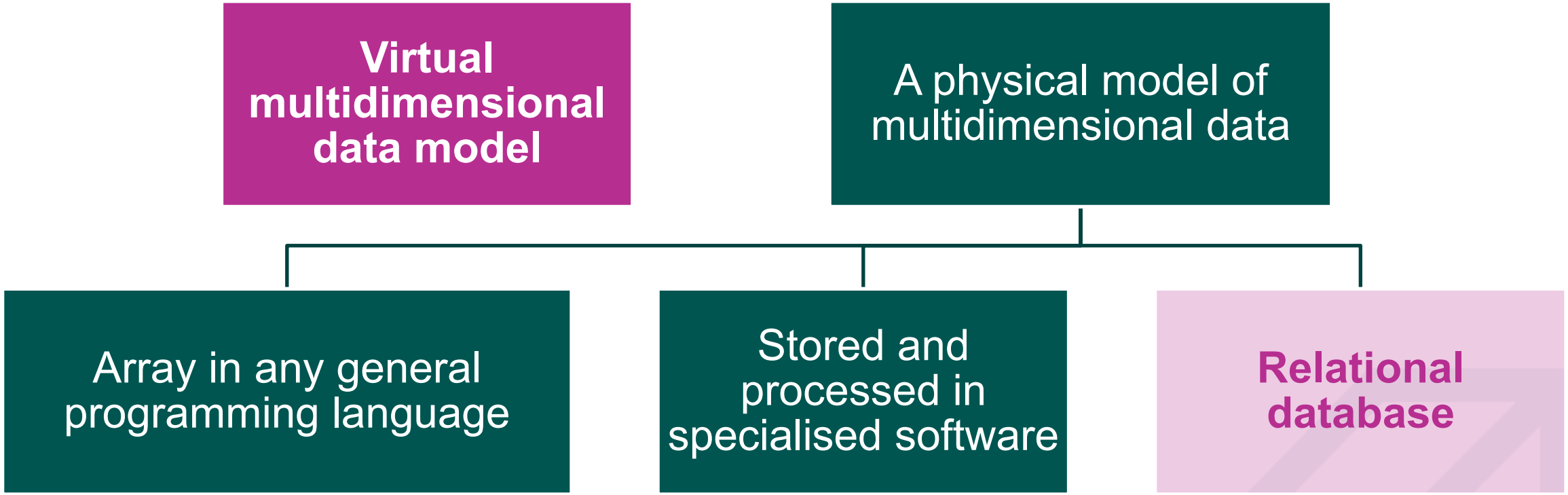


Hierarchies visualised





Where have you seen such a data structure?



Virtual multidimensional data model

- It is not always necessary to store such a data model physically - for example, if this is necessary **for processing in application** or **demonstration**
- Several options are possible:
  - PIVOT
  - GROUP BY extensions
  - MODEL

GROUP BY extensions

Multidimensional grouping

- In principle, it is possible to get a virtual fact table using **GROUP BY** with different attributes
- However, it is also possible to retrieve the results of aggregation functions at different cross-sections:
  - **ROLLUP** extension - retrieves only the results of aggregation functions based on the order of the fields to be grouped
  - **CUBE** extension - retrieves the results of aggregation functions for all combinations of field values

ROLLUP and CUBE extensions

```
SELECT destination, bus_number, driver, count(trip_number) AS COUNT
FROM trips
GROUP BY destination, bus_number, driver;
```

15 rows

```
SELECT destination, bus_number, driver, count(trip_number)
FROM trips
GROUP BY ROLLUP (destination, bus_number, driver);
```

30 rows

```
SELECT destination, bus_number, driver, count(trip_number)
FROM trips
GROUP BY CUBE (destination, bus_number, driver);
```

60 rows

Presentation and processing of results*

- The GROUPING function allows you to get a bitmap for each row based on whether the values are grouped
- DECODE allows you to present the result

```
SELECT
    DECODE(GROUPING(destination),1,'Total', destination) as d,
    DECODE(GROUPING(bus_number),1,'Total', bus_number) as bn,
    DECODE(GROUPING(driver),1,'Total', driver) as dr,
    count(trip_number)
FROM trips
GROUP BY CUBE (destination, bus_number, driver)
HAVING GROUPING(destination) = 0;
```

Prague	D474B453	98614209810	1
Berlin	Total	Total	5
Madrid	1L1K3	Total	2
Madrid	Total	Total	2
Poznan	5Y573M5	Total	3

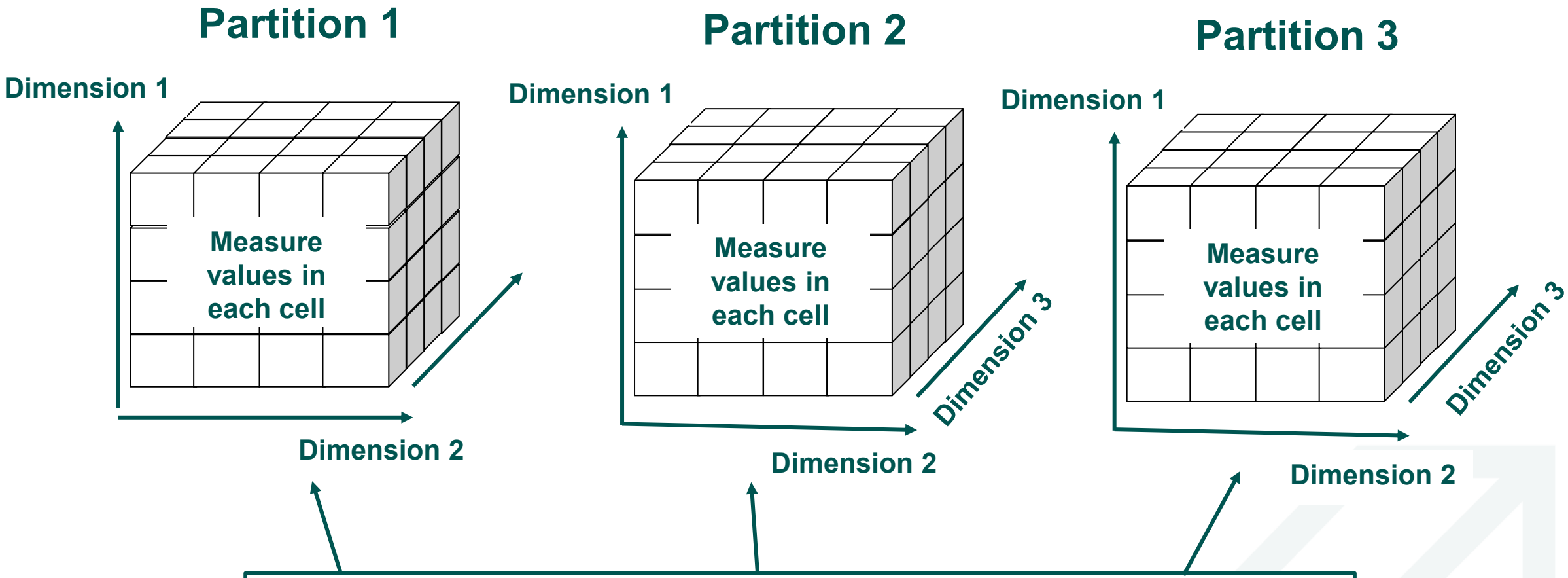
* slide has an explanatory script

MODEL clause

Creating virtual multidimensional data arrays

- The SQL Model clause **creates a virtual multidimensional model (or models)** from the query results and then **uses rules** (or formulas, routines) in this model to calculate new values
- The amount of data is limited only by memory resources
- Virtual models can be shared **across workgroups**, ensuring that calculations are consistent across applications

Building a virtual multidimensional model



Rules

for recalculating cube cell values and creating new virtual cells

Components of a virtual data store

- **Partitions** define logical blocks of a result set that are independent of each other - used for paralelisation
- **Dimensions** define a multi-dimensional virtual cube and are used to identify cells; **a full combination of dimensions need to only identify one cell**
- **Measures** contain numerical values.

- Each cell can be accessed by specifying **the full** combination **of dimensions**
- **Each partition** can have a cell corresponding to the relevant combination of dimensions.

SQL MODEL clause structure for building a virtual multidimensional model

```
SELECT ... FROM ... WHERE
MODEL
    [PARTITION BY (<fields>)]
     DIMENSION BY (<fields>)
    MEASURES (<fields>)
    RULES (<rule>, <rule>,.., <rule>)
```

- **PARTITION BY** – values of these fields will be basis for multiple virtual cubes
- **DIMENSION BY** – values of these fields will become analysis factors, i.e. dimensions
- **MEASURES** – indicates which fields will become measures
- **RULES** – specify the rules

Simple MODEL example*

```
CREATE OR REPLACE VIEW trip_statistics_model AS
(SELECT driver, destination, bus_number, count(trip_number) as trip_count
 FROM trips
 GROUP BY driver, destination, bus_number);

SELECT *
FROM trip_statistics_model
MODEL
DIMENSION BY (destination, driver, bus_number)
MEASURES (trip_count)
RULES();
```

* slide has an explanatory script

MODEL example with partitions*

```
SELECT *
FROM salaries s JOIN bus_drivers d
ON s.driver_code = d.personal_code
MODEL
PARTITION BY (month)
DIMENSION BY (year, driver_last_name)
MEASURES (paid_salary)
RULES()
ORDER BY year;
```

* slide has an explanatory script

Rules

- In principle, we can get similar result with GROUP BY (apart from paralelization possibilities)
- The main idea of the MODEL clause is the ability to calculate and add new virtual cells, thus performing the calculations on the database side
- This is done by using rules
- Rules provide access to and update cell values by explicitly specifying dimension values

Types of rules

- Single cell reference
  - paid_salary[2023,'Spanovskis','JAN'] = paid_salary[2023,'Spanovskis','JAN']*1.1
- Multi-cell reference
  - paid_salary[year > 2021,'Bumbiere','DEC'] = paid_salary[2023,'Bumbiere','JAN']*1.1
- Using multiple values on both sides
  - paid_salary[2023,'Pauls','JAN'] = MAX(paid_salary)[2023,'Spanovskis', month BETWEEN 'NOV' AND 'DEC']
  - paid_salary[2022, 'Siksna', month IN ('NOV', 'DEC')] = paid_salary[2022, 'Freidenfelds', CV(month)]*1.5
- "Slices" reference
  - paid_salary[2022, driver_last_name IS ANY, 'OCT'] = 10000
- Iterative laws are also possible

Example with rules*

```
SELECT *
FROM salaries s JOIN bus_drivers d
ON s.driver_code = d.personal_code
MODEL
DIMENSION BY (year, driver_last_name, month)
MEASURES (paid_salary)
RULES(
    paid_salary[2023,'Spanovskis','JAN'] = paid_salary[2023,'Spanovskis','JAN']*1.1,
    paid_salary[year > 2021,'Bumbiere','DEC'] = paid_salary[2023,'Bumbiere','JAN']*1.1,
    paid_salary[2023,'Pauls','JAN'] = MAX(paid_salary)[2023,'Spanovskis', month BETWEEN 'NOV' AND 'DEC'],
    paid_salary[2022, 'Siksna', month IN ('NOV', 'DEC')] = paid_salary[2022, 'Freidenfelds', CV(month)]*1.5,
    paid_salary[2022, driver_last_name IS ANY, 'OCT'] = 10000
)
ORDER BY year;
```

* slide has an explanatory script

Q&A